

C# Photo Editing Engine

Architecture, Real-Time GPU Pipelines, Preset Systems & HDR Merging

ENGINEERING TECHNICAL REFERENCE

1. Primary Core Feature Specifications

To build a focused, production-grade photo editor without bloated scope, implementation should prioritize the vital adjustment primitives organized into four operational domains:

Global Tone & Canvas

- **Exposure / Brightness:** Linear gain / EV scaling.
- **Contrast:** Midtone S-curve slope modulation.
- **Highlights & Shadows:** Tonal lift/recovery in upper & lower luminance bands.
- **Whites & Blacks:** Dynamic range boundary clipping pins.
- **Live Histogram:** RGB/luminance graph with over/underexposure clipping masks.

Color & White Balance

- **Temperature & Tint:** Amber-Blue (Kelvin) and Green-Magenta shift.
- **Vibrance & Saturation:** Perceptual/muted-weighted boost vs. uniform gain.
- **HSL / Color Mixer:** Per-channel Hue, Saturation, and Luminance adjustment for 6 primary/secondary colors.

Detail & Geometry

- **Sharpness & Noise:** High-pass / unsharp mask and luminance/chrominance smoothing.
- **Crop & Aspect:** Freeform and standard ratio rectangular bounding boxes (1:1, 4:3, 16:9).
- **Straighten & Flip:** Fine continuous angle rotation with auto-crop; 90° rotation and horizontal/vertical mirroring.

Workflow & Pipeline Foundation

- **Viewport Zoom & Pan:** 1:1 pixel inspection and canvas navigation.
- **Before / After Toggle:** Split-view or press-and-hold original reference.
- **Linear Undo/Redo:** State history stack.
- **Export Engine:** JPEG, PNG, WebP, TIFF with configurable quality and resizing.

2. Recommended Open-Source C# Stack

Modern photo editing engines avoid using a single library for everything. The ideal architecture cleanly isolates raw sensor decoding, interactive viewport processing, and file export serialization.

• RAW Sensor Decoding & Metadata: [Sdcb.LibRaw](#)

Direct C# wrapper around [LibRaw](#). Handles Bayer demosaicing, black/white point calibration, camera color matrices, and EXIF extraction across Nikon (.NEF), Canon (.CR2/.CR3), Sony (.ARW), and Adobe DNG. Delivers 16-bit linear or 32-bit floating-point image buffers.

• Interactive Viewport & Shader Pipeline: [SkiaSharp](#) (SkSL)

Cross-platform 2D engine backed by Google Skia with hardware acceleration across OpenGL, Vulkan, and Metal. Supports custom GPU fragment shaders via [SKRuntimeEffect](#) (SkSL), allowing sub-millisecond slider updates directly on GPU textures.

- **Headless Encoding & Offline Batch Processing:** [SixLabors.ImageSharp](#)
Fully managed, GC-optimized library for ICC color profile embedding, metadata retention, and headless file generation (JPEG, WebP, PNG) during disk export.
- **Compute Acceleration (Alternative):** [ComputeSharp](#)
For Windows DirectX 12 targets, allows writing HLSL compute shaders directly in C# for heavy multi-pass operations (bilateral denoising, non-local means).

3. Performance Architecture: SkiaSharp vs. ImageSharp vs. darktable

3.1. Skia GPU Runtime vs. CPU-Based Processing

Dimension	SkiaSharp + SkSL (GPU Shaders)	SixLabors.ImageSharp (CPU)
Execution Model	Massively parallel SIMD across GPU cores.	Multi-threaded CPU task pool across CPU cores.
Slider Interaction Latency	< 5–16 ms (Consistent 60–144 FPS).	150–1500+ ms per frame on 24–45MP images.
Slider Update Overhead	Passes small uniform float structs (~few bytes).	Re-traverses and calculates 24M–45M pixel values.
Memory Churn	Zero allocation per frame; textures stay in VRAM.	Substantial GC pressure without aggressive pooling.
Viewport Transformations	Hardware matrix multiplication (Zero cost).	Requires explicit resampling and array slicing.

3.2. Why darktable Feels Sluggish (and Why SkSL Outperforms It in UI)

darktable uses a rigorous, scientific **scene-referred** node-based graph called the *pixelpipe*. When you move a slider in darktable, it re-evaluates 20–30 isolated OpenCL modules sequentially on unbounded 32-bit floating-point buffers. While physically accurate, this creates pipeline synchronization stalls and fractional-second hitches.

By contrast, an interactive SkSL architecture decodes the image once, loads it into a high-precision texture, and executes a **fused, single-pass fragment shader** only across the screen-space pixels currently visible on the viewport canvas. Sliders adjust uniforms, producing immediate, zero-latency feedback comparable to Adobe Lightroom.

4. Portable Preset Architecture (Lightroom vs. darktable)

Core Principle: Display-Referred vs. Scene-Referred Presets

Presets in darktable fail across different images because they apply absolute mathematical transformations to unbounded raw scene light. Lightroom presets succeed because they apply **relative aesthetic deltas** onto a normalized perceptual space.

To implement reliable, portable presets in your application:

1. **Pre-Normalization Anchor:** Extract sensor white balance and compute an exposure baseline (mapping image midtones to an 18% gray baseline / ~0.18 linear) before applying user styles.

2. **Delta Uniform Serialization:** Store presets as lightweight JSON files defining offsets, not absolute values. Keep white balance and overall exposure optional.
3. **3D LUT Integration:** For vintage film emulation or cinematic grades, support standard `.cube` 3D lookup tables unwrapped into 2D GPU textures and sampled in your SkSL shader.

```
// Sample Preset Data Structure (JSON)
{
  "presetName": "Warm Editorial",
  "version": 1,
  "settings": {
    "temperatureDelta": 120.0,
    "contrast": 1.12,
    "highlights": -0.28,
    "shadows": 0.18,
    "vibrance": 0.14,
    "hsl": {
      "orange": { "h": -4.0, "s": 8.0, "l": 5.0 },
      "blue": { "h": 6.0, "s": -15.0, "l": -8.0 }
    }
  }
}
```

5. High Dynamic Range (HDR) Exposure Merging in C#

Merging multiple bracketed exposures into a single 32-bit floating-point radiance map can be accomplished via two methods:

Method A: OpenCvSharp4 (Computer Vision Engine)

OpenCV provides battle-tested implementations of camera alignment and radiance estimation:

- `AlignMTB` : Ward's Median Threshold Bitmap algorithm to align handheld bracketed exposures without homography distortions.
- `CalibrateDebevec` : Recovers the Camera Response Function (CRF) to map non-linear pixel values back to scene radiance.
- `MergeDebevec` : Accumulates aligned shots into a 32-bit floating-point radiance matrix (`CV_32FC3`).
- `MergeMertens` : Alternative multi-exposure fusion when true 32-bit HDR radiance is not required, producing an 8/16-bit tone-fused output directly.

Method B: Native Linear RAW Merging

Because RAW files decoded via `Sdcb.LibRaw` are already linear, CRF estimation is bypassed. Merging consists of:

1. **Image Registration:** Align frames via phase correlation or MTB pyramids.
2. **Exposure Normalization:** Scale pixels by exposure duration ($\text{ext}\{\text{Radiance}\}_i = \text{rac}\{P_i\}\{\Delta t_i\}$).
3. **Weighted Hat-Accumulation:** Apply a bell-curve weighting function ($w(z)$) centered around 0.5 midtones to penalize clipped highlights and noisy shadows.
4. **Motion De-Ghosting:** Establish the median exposure as spatial ground-truth. Detect motion via local patch difference masks, zeroing the weight of moving objects on secondary frames.
5. **Skia Upload:** Pass the final merged float buffer into an `SKColorType.RgbaF32` texture and apply an interactive tone-mapping operator (e.g., Reinhard, ACES) within your SkSL viewport shader.